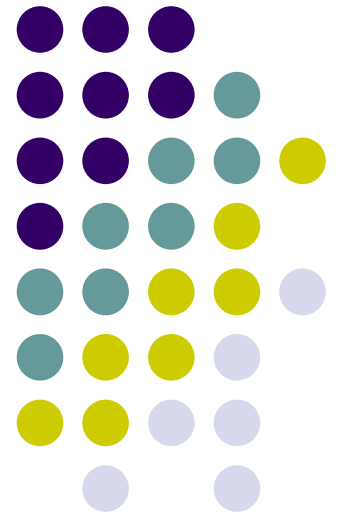
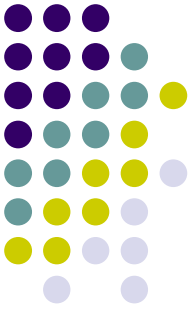


Hubs, Bridges & Spanning Trees

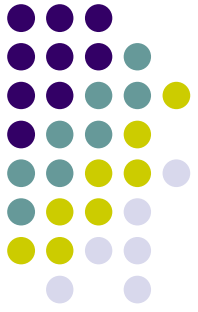
-Connecting datalinks





Where are we?

- The big picture:
 - Saw physical layer, datalink layer
 - So far: datalink provides simple, quite effective communication: Local Area Networks



Interconnecting LANs

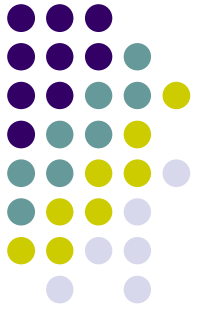
Q: Why not just one big LAN?

A: LAN do not scale well

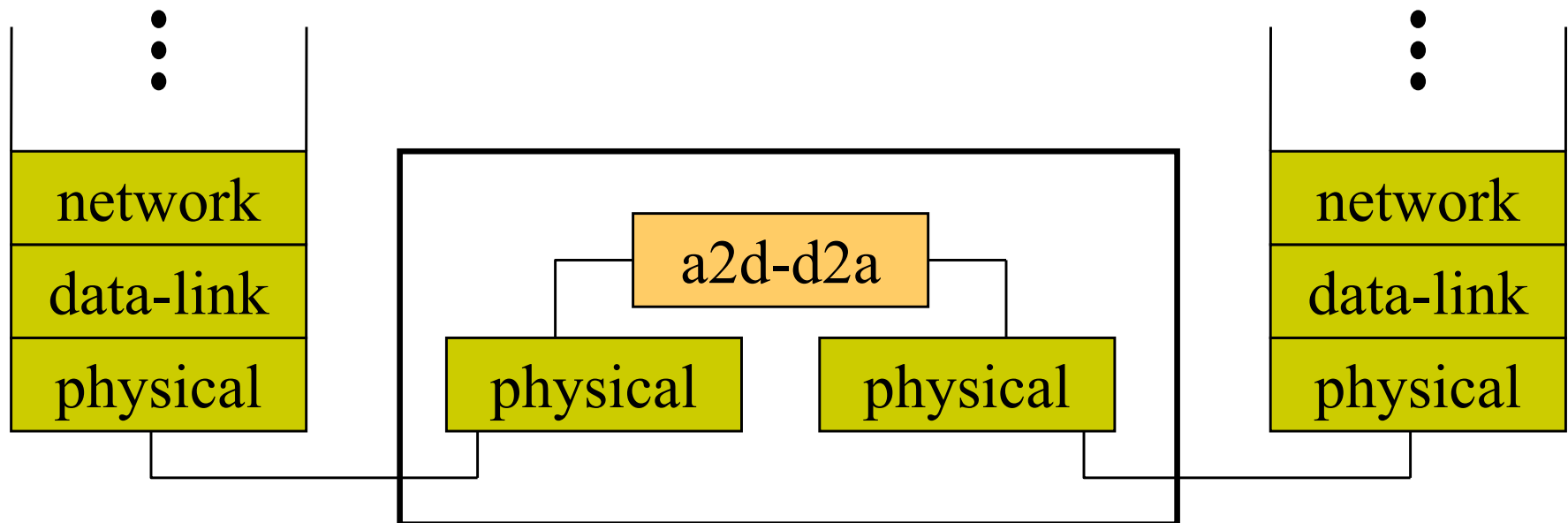
In more detail:

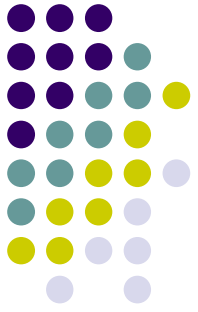
- Protocol correctness:
 - (Length) 802.3 (Ethernet) specifies maximum cable length
- Protocol performance:
 - (Width) All stations share one large "collision domain", number of stations is limited

Partial Solution 1: Repeaters



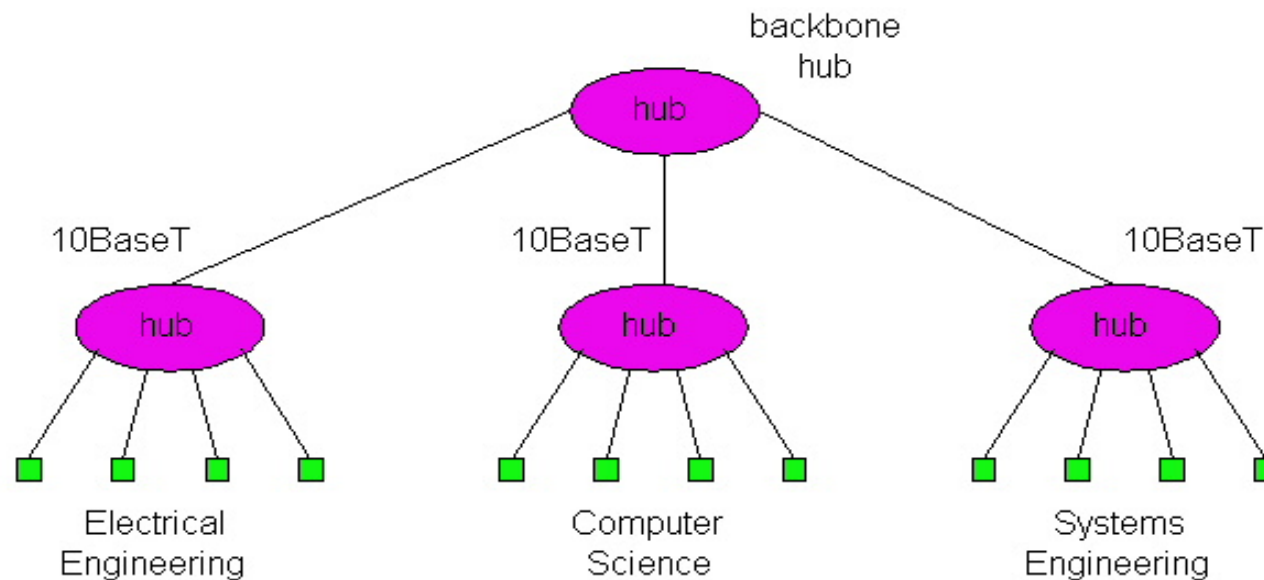
Goal: extend physical layer length
Means: recreate the signal



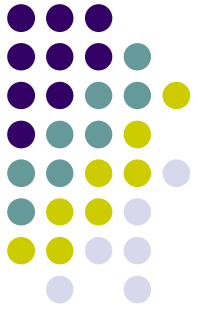


Generalization: Hubs

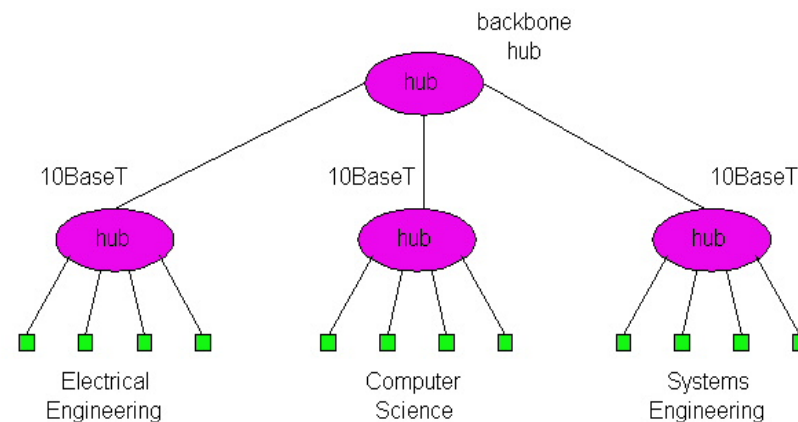
- Repeaters with many ports (legs): repeat bits received from one port to all other ports
- Hubs can be arranged in a **hierarchy** (or multi-tier design), with **backbone** hub at its top

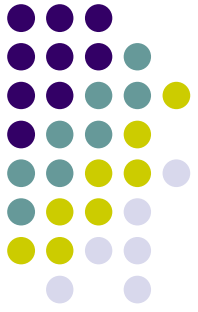


Hub Properties



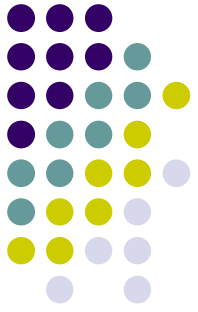
- Hub Advantages:
 - Simple, inexpensive device
 - extends maximum distance between node pairs (100m per Hub)
 - Multi-tier provides graceful degradation: portions of the LAN continue to operate if one hub malfunctions





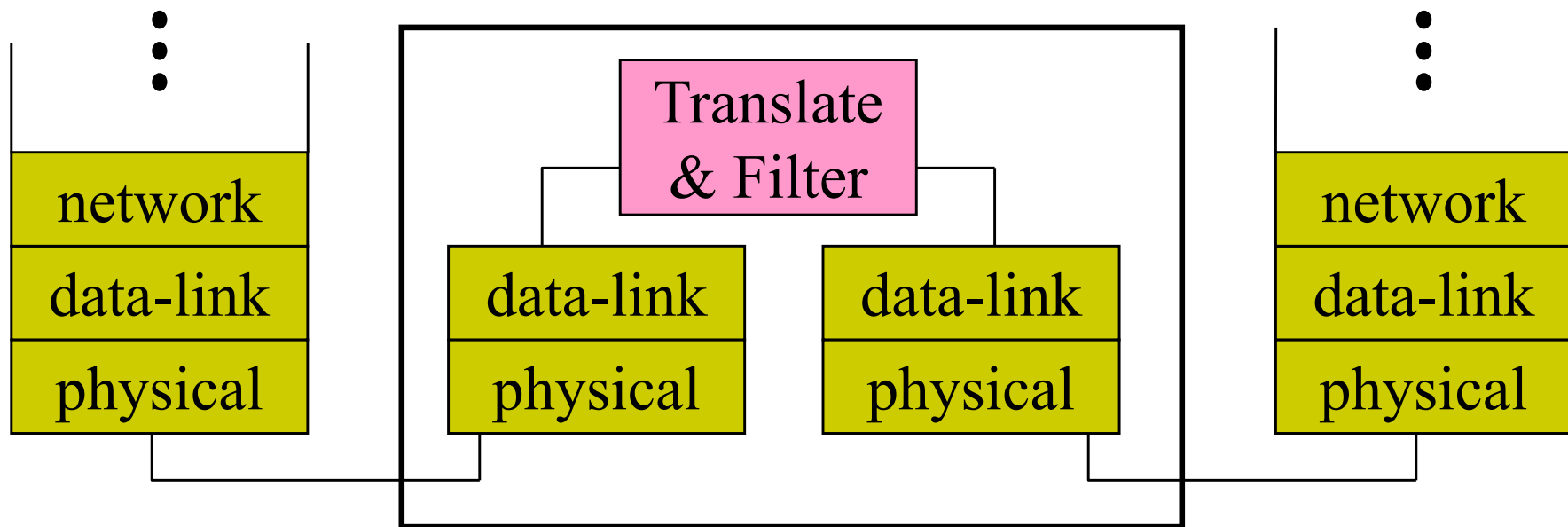
Hub limitations

- LAN segment: looks like a single medium (possible repeaters)
- A segment is a single **collision domain**: a node may collide with any other same-segment node
 - ➔ no increase in total throughput
 - Multi-tier throughput same as single segment throughput
- Individual LAN restrictions pose limits on number of nodes in same collision domain and on total allowed geographical coverage



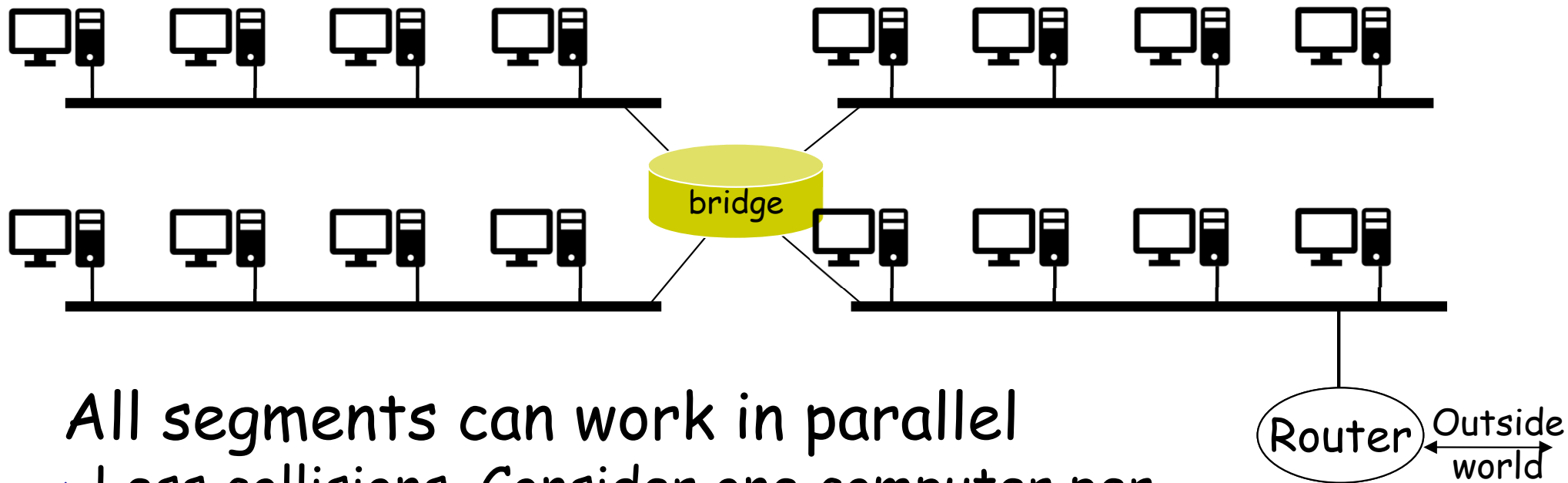
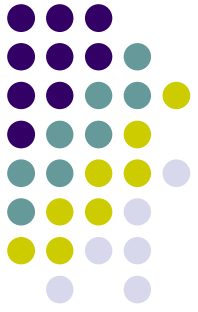
Next (big) step: Bridges

- Connect LANs at the data-link layer



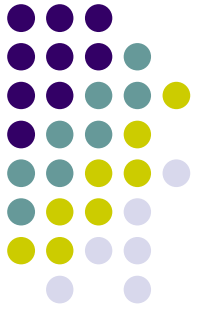
- Isolate collision domains: store and forward!
- Forwarding is selective: do not always flood.

Ethernet bridges (aka switches)



All segments can work in parallel

- ❖ Less collisions. Consider one computer per port
- ❖ More capacity: a switch can forward multiple frames to different computers at the same time.



Ethernet Bridge Operation

Idea: maintain a local forwarding table associating (some) Ethernet addresses with ports.

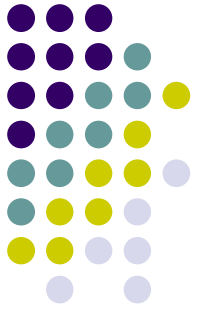
Table is built (learned) automatically!

Algorithm:

SA = source address

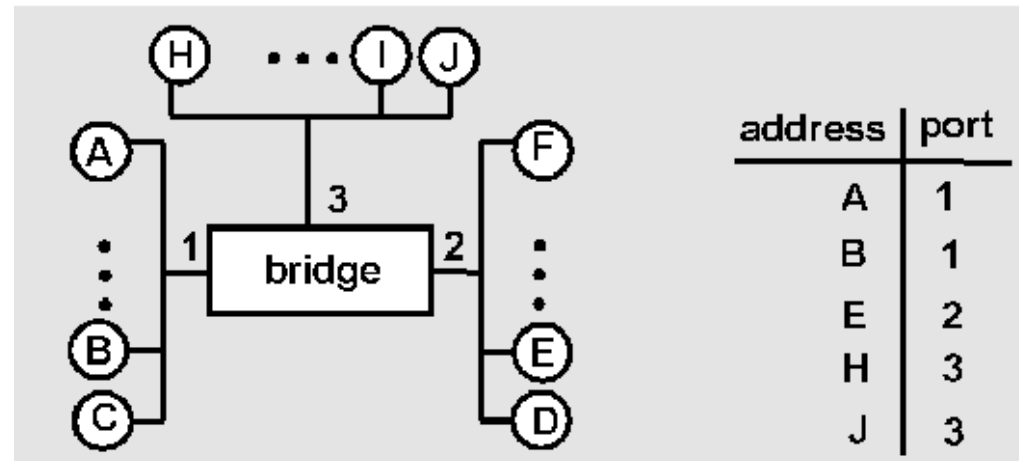
DA = destination address

- Whenever a frame from *SA* to *DA* arrives on port *p*:
 1. Enter (*SA*, *p*) into table with expiration time
 2. If *DA* is a broadcast address, or if *DA* is NOT in table: broadcast frame to all ports except for port *p*.
 3. Otherwise (*DA* is in table and not broadcast): forward frame to port indicated by table.
- Ongoing in background: Throw out expired entries



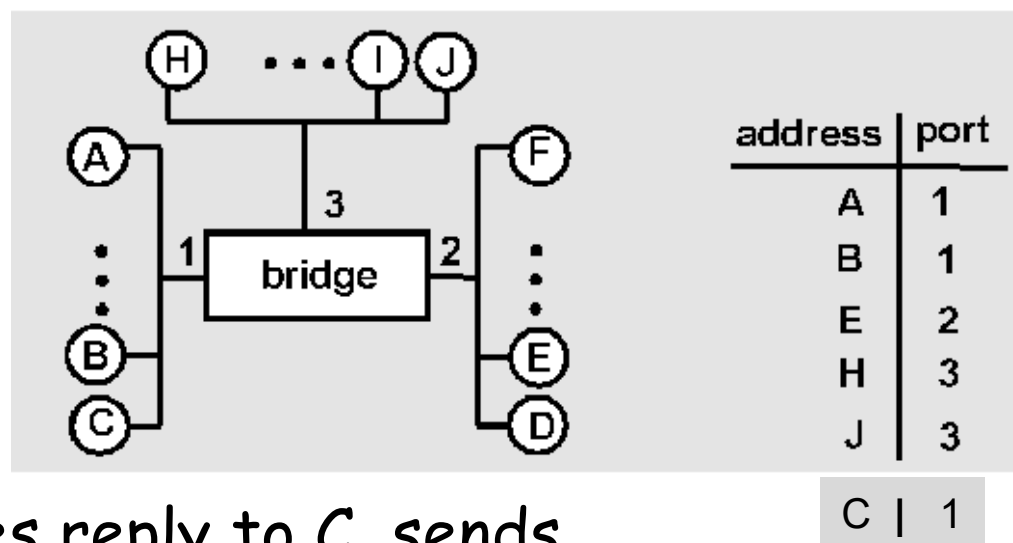
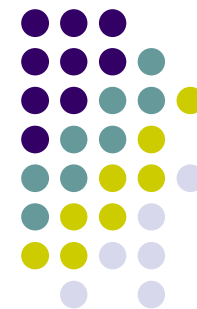
Bridge Learning: example

Suppose *C* sends frame to *D* and *D* replies back with frame to *C*



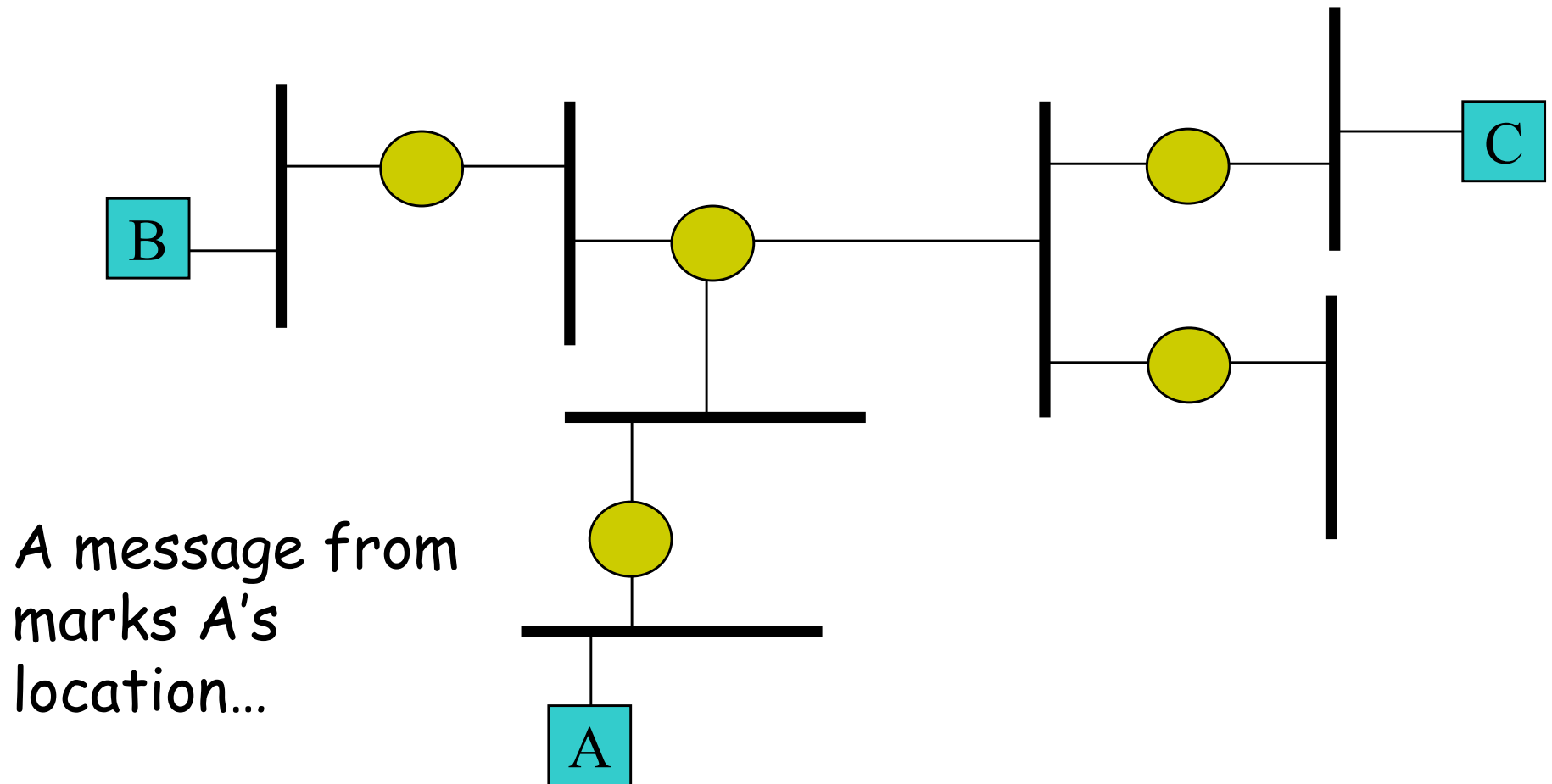
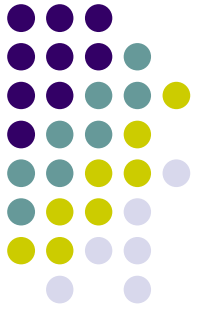
- *C* sends frame, bridge has no info about *D*, so floods to both LANs
 - bridge notes that *C* is on port 1
 - frame ignored on upper LAN
 - frame received by *D*

Bridge Learning: example

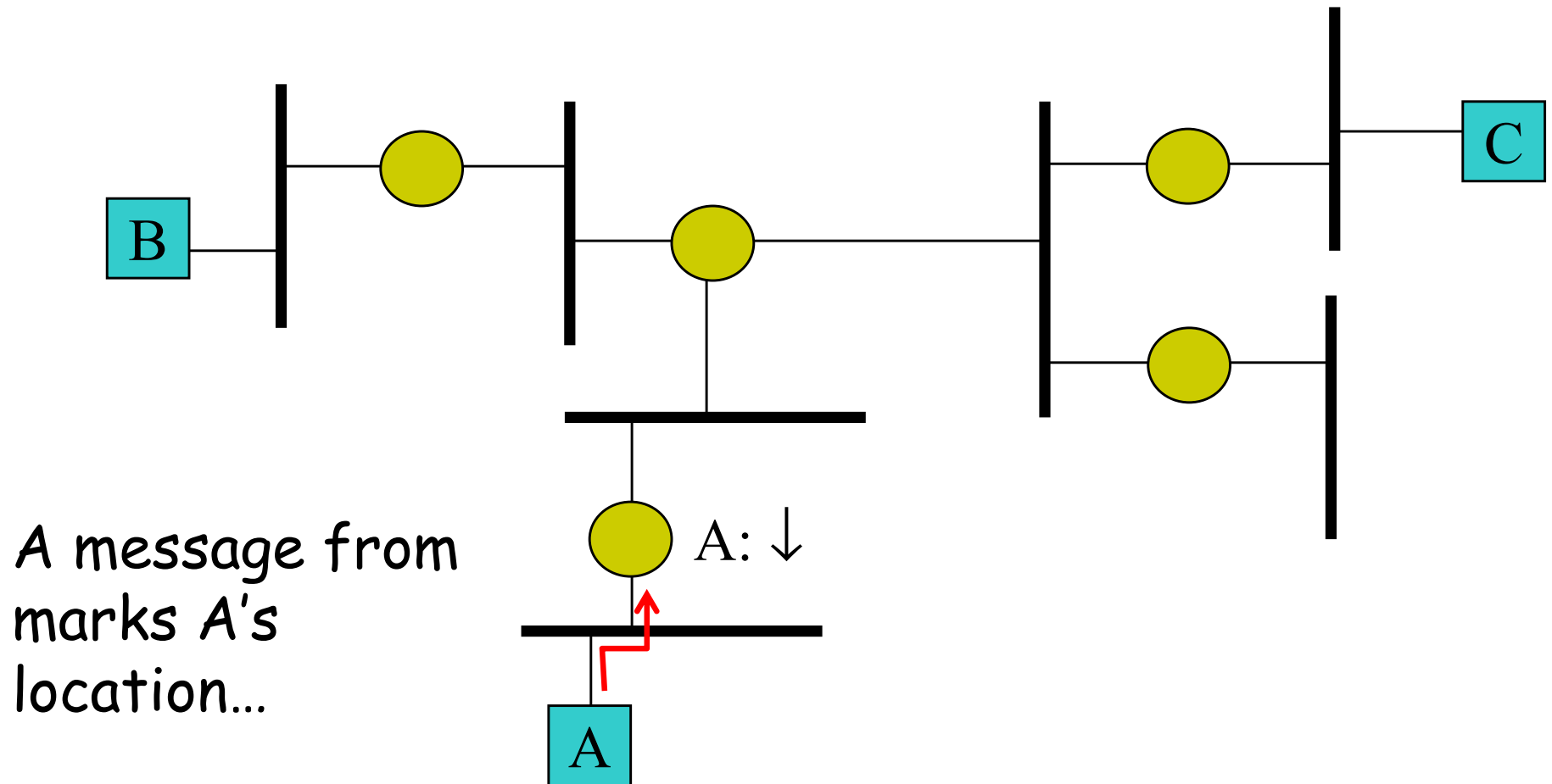
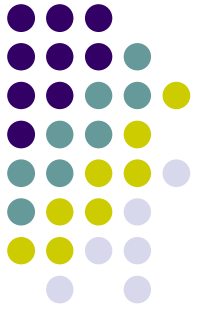


- D generates reply to C, sends
 - bridge sees frame from D
 - bridge notes that D is on interface 2
 - bridge knows C on interface 1, so *selectively* forwards frame out via interface 1

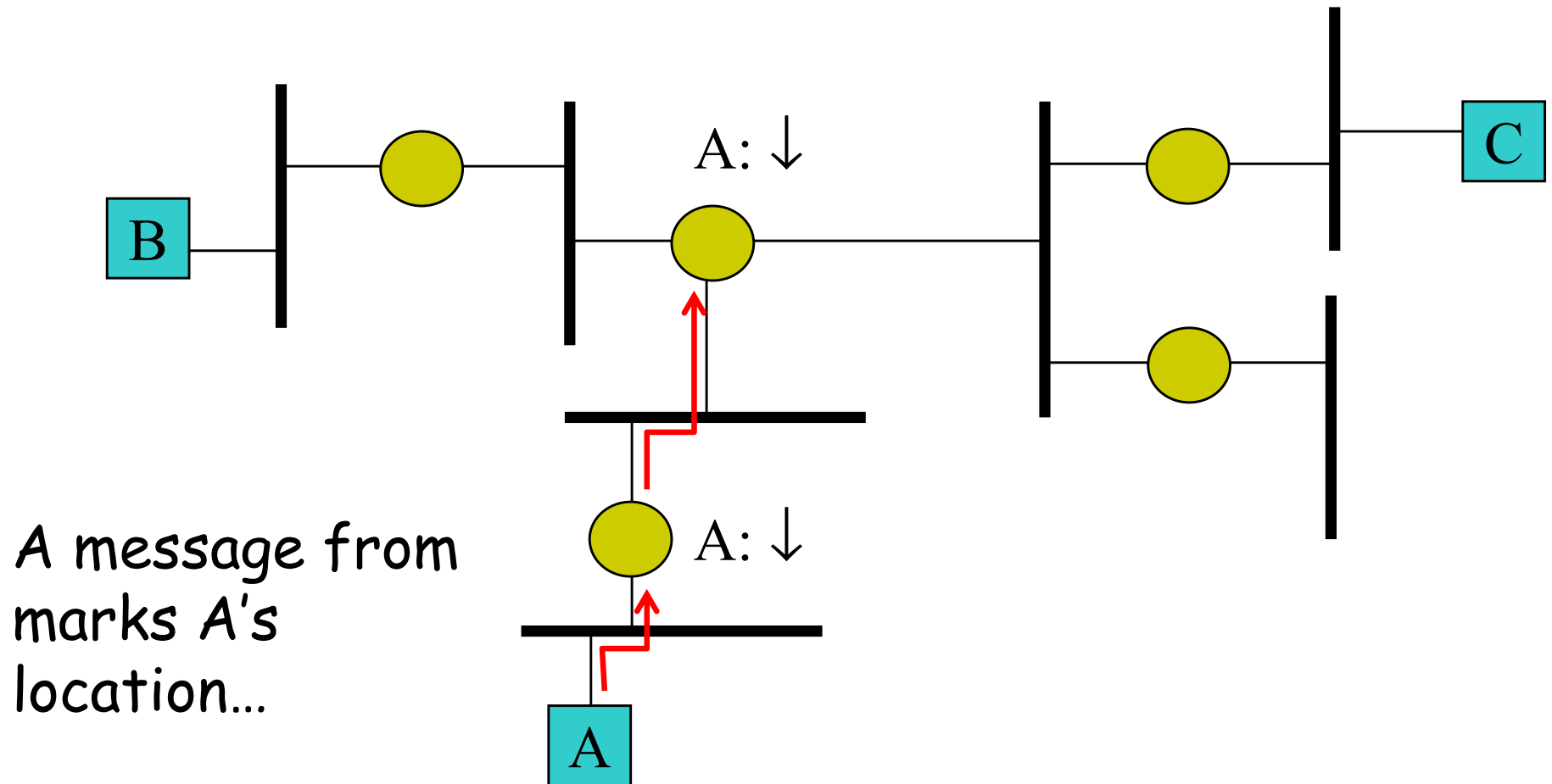
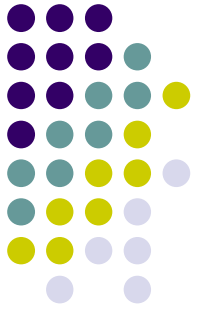
Operation in trees



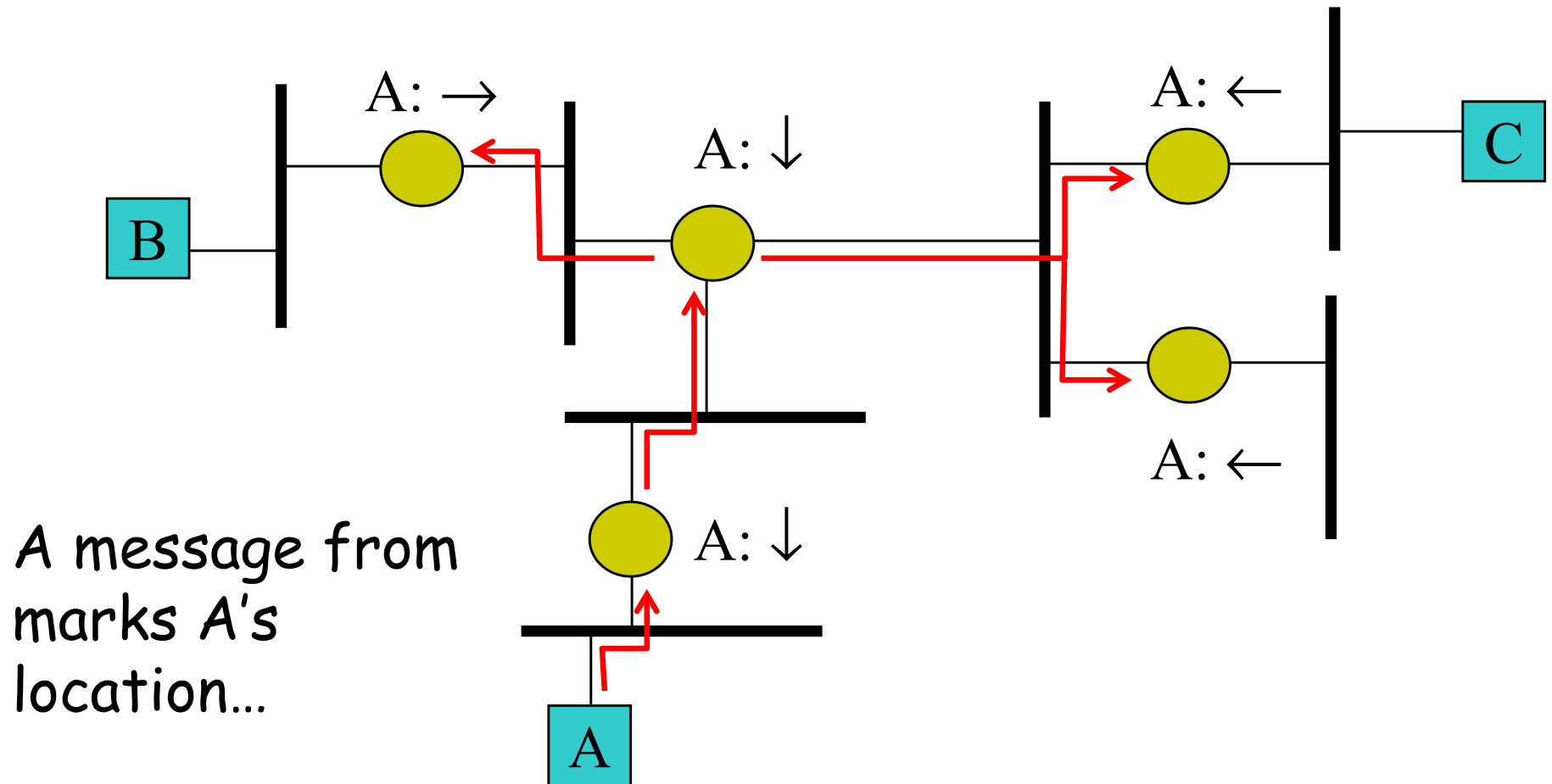
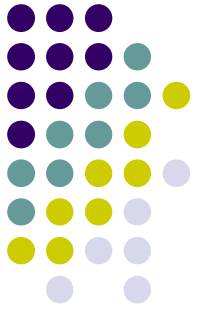
Operation in trees



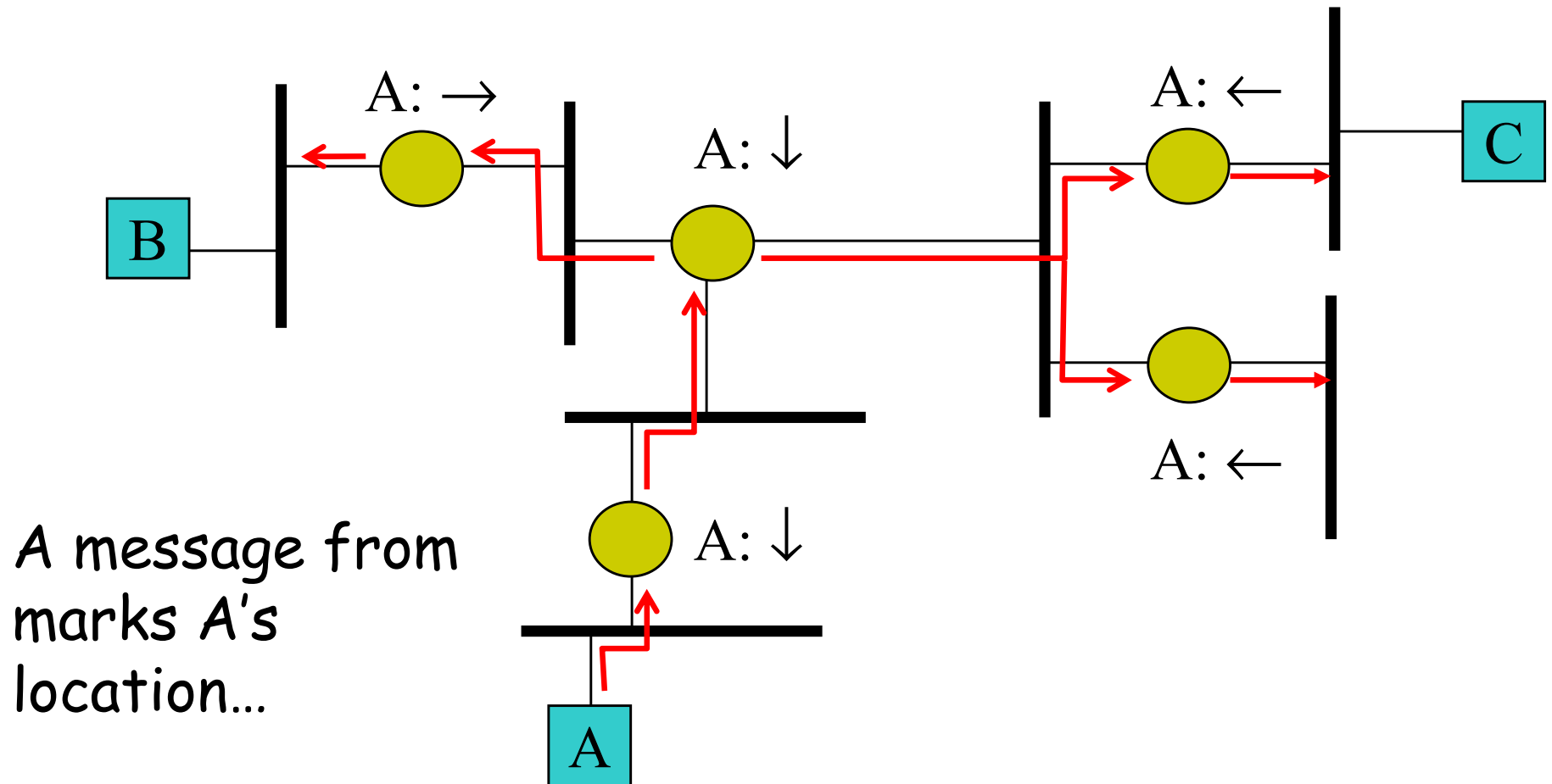
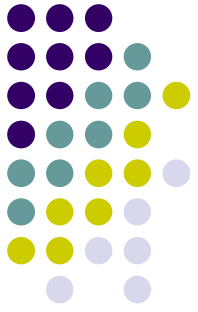
Operation in trees



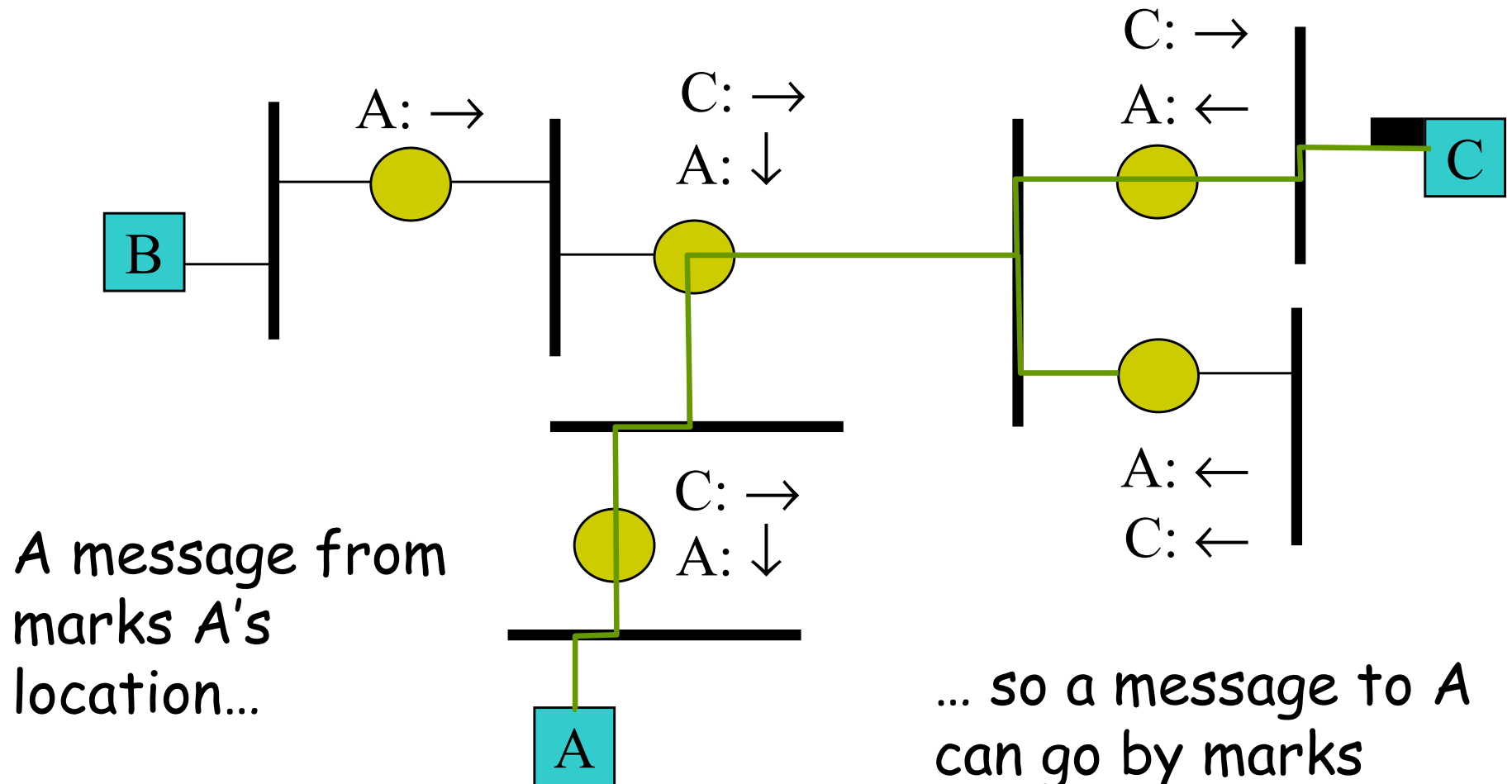
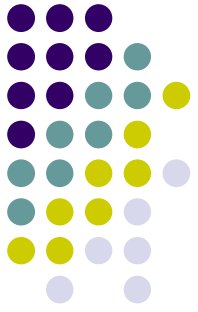
Operation in trees



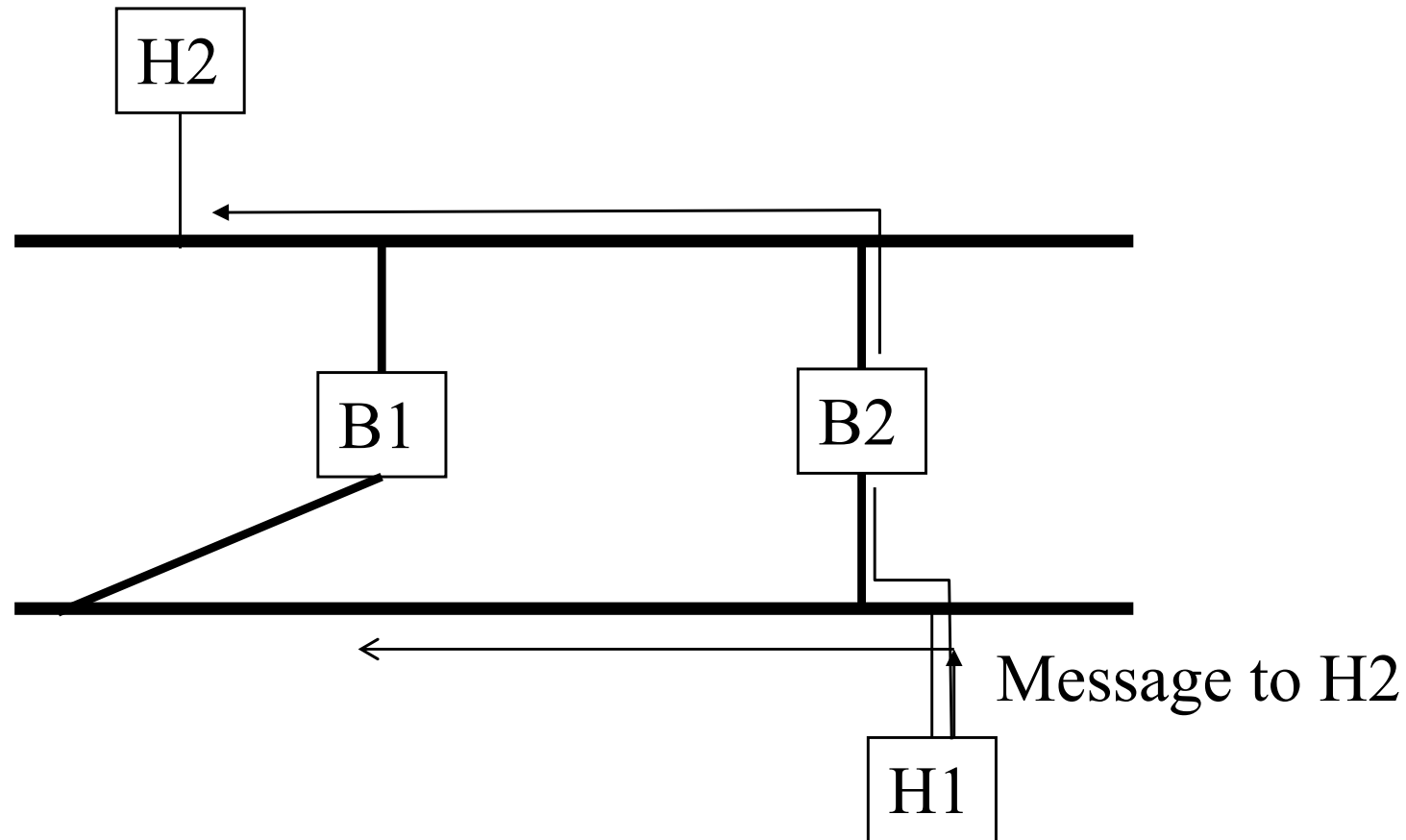
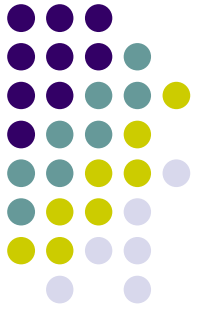
Operation in trees



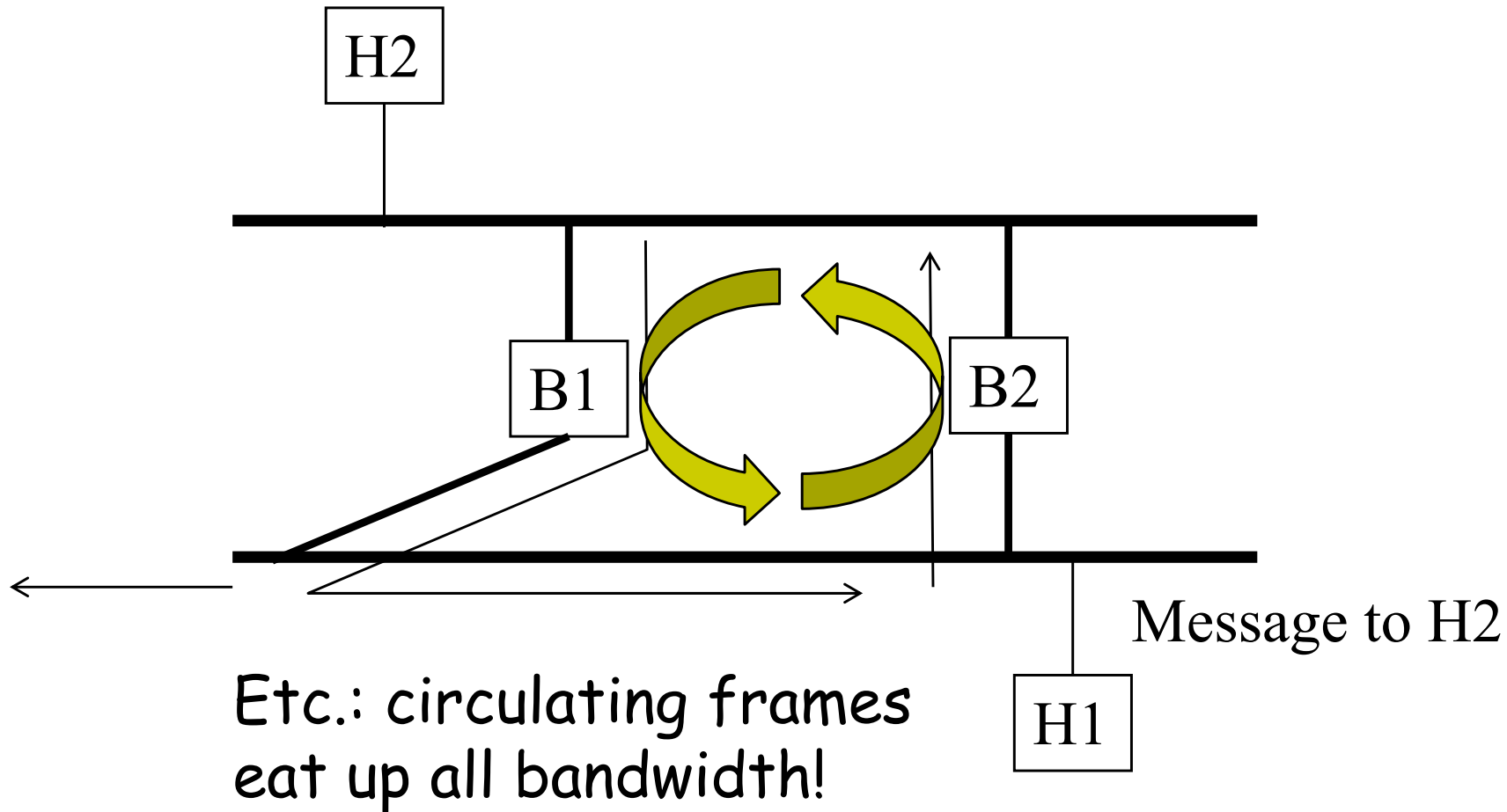
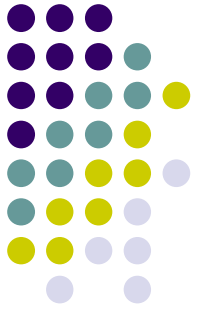
Operation in trees

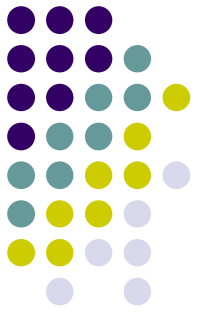


Problem with bridges: Loops



Problem with bridges: Loops



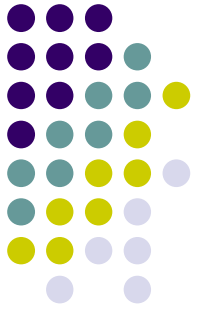


Trees are good, loops are bad...

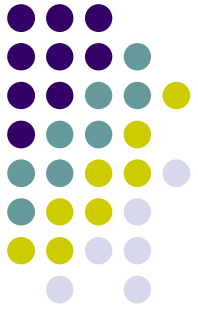
- Actually: redundant paths are good!
- But we don't want to use them when not needed
 - Losing bandwidth, earning simplicity
- We can say that loops are **verboten**!
 - (sounds more convincing in German)
 - But can we help it?
- Solution: a protocol that will disable some connections to make the topology loop-free

Spanning tree protocol!

How to construct a spanning tree?



- Bridges run a distributed spanning tree algorithm
 - Select what ports (and bridges) should actively forward frames
- Standardized in IEEE 802.1 specification



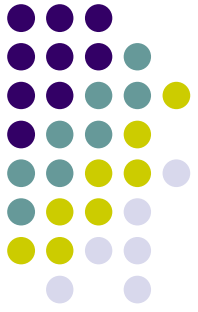
Overview of STP

We make a series of simplifications:

- Build a ST of **bridges** (not **segments**)
- Assume that we are given a **root** bridge

So we solve in bottom-up order:

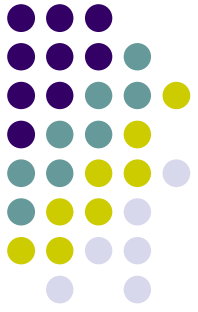
1. Agree on a root bridge
2. Given a root, compute a ST of bridges
3. Given a ST of bridges, compute an ST of LAN segments



1. Choosing a root bridge

- Assume each bridge has a unique identifier
- Each bridge remembers best ID seen so far (variable: **my_root_ID**)
- Periodically, send **my_root_ID** to all neighbors ("flooding")
- When receiving ID, adopt if smaller than **my_root_ID**
- Faults?!

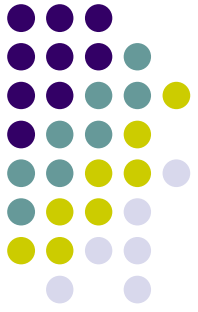
2. Given a root, compute ST



Idea: each node finds its shortest path to the root → shortest paths tree

Output: At each node, parent pointer (and distance)

How: **Bellman-Ford** algorithm

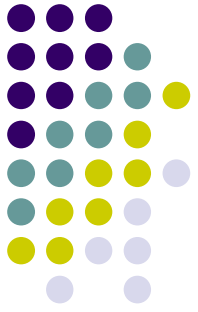


Distributed Bellman-Ford

Assumption: There is a unique root node s

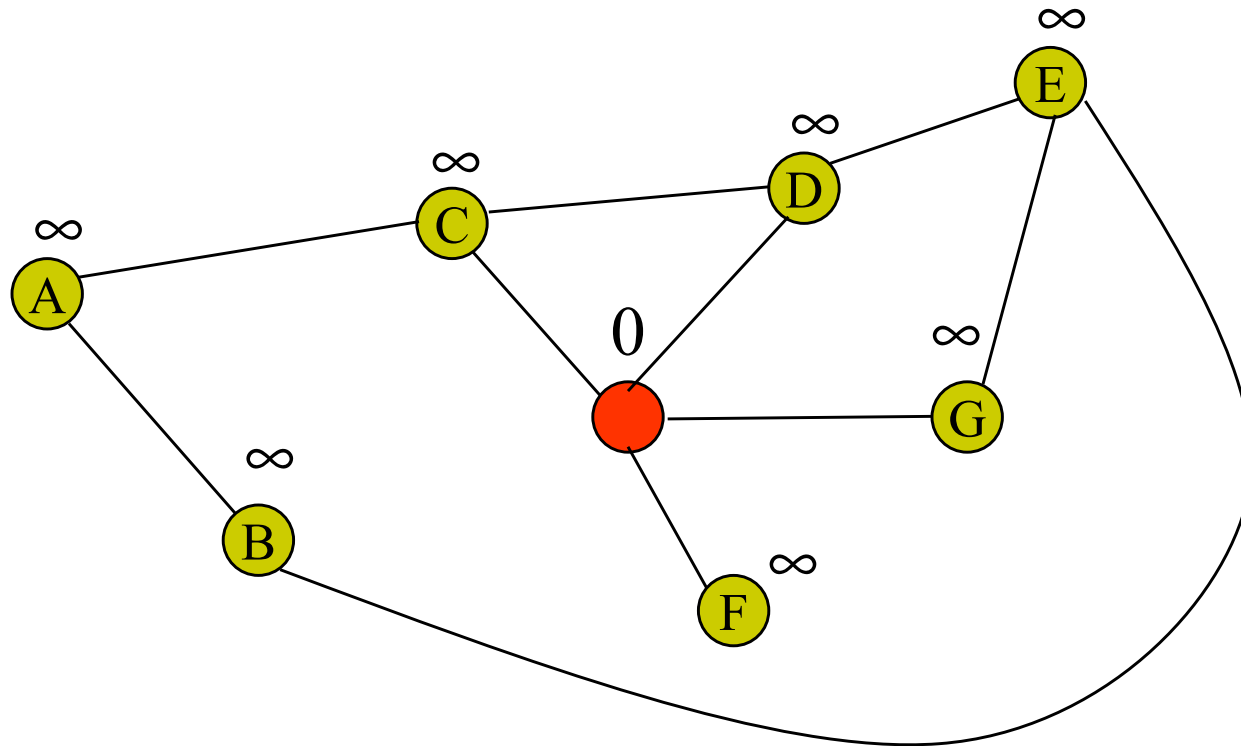
Idea: Each node, periodically, tells all its neighbors what is its **distance from s**

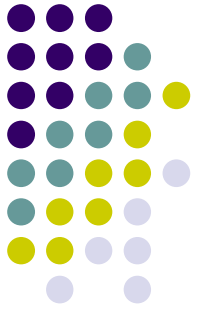
- s : easy... $\text{dist}_s = 0$ always.
- Node $v \neq s$:
 1. Compute cost of going through neighbor u :
 $\text{cost}_v(u) := \text{dist}_u + 1$
 2. Choose smallest:
 $\text{dist}_v := \min\{\text{cost}_v(u) \mid u \text{ is a neighbor of } v\}$
- **Root Port:** leading to least-cost neighbor



Why does this work?

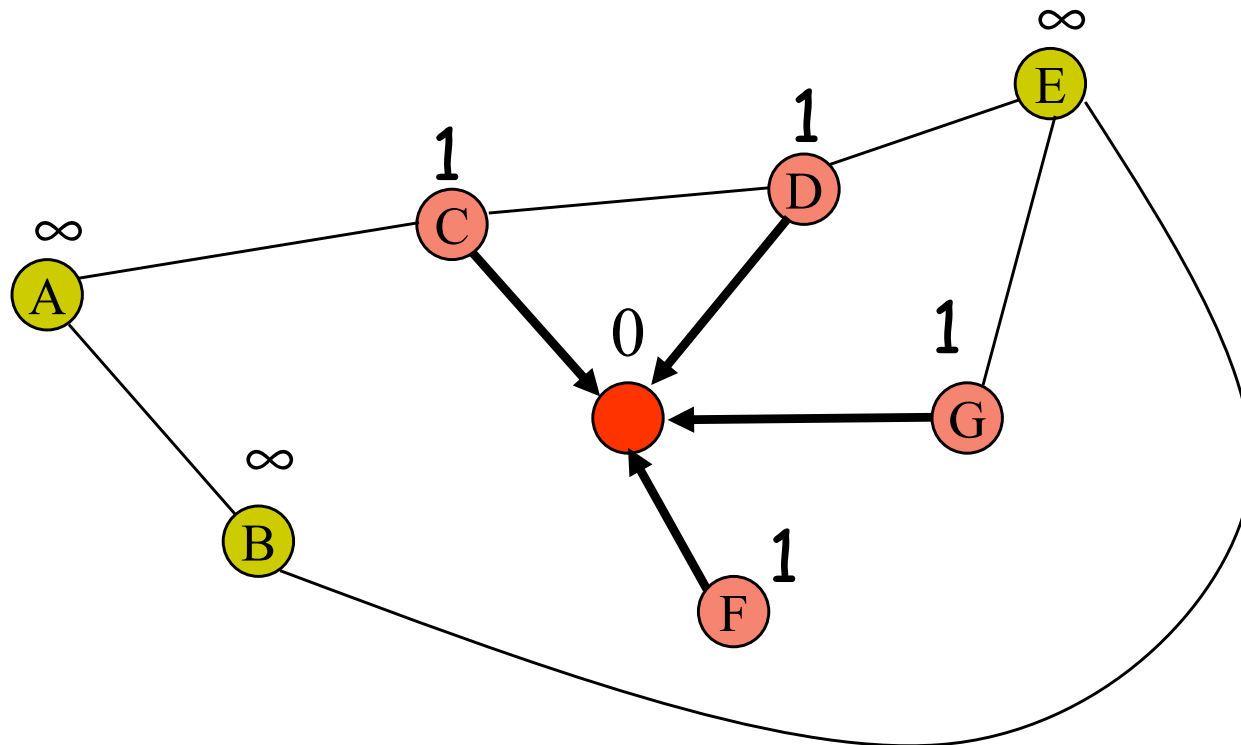
- Suppose all nodes start with distance ∞ , and suppose that updates are sent every time unit.

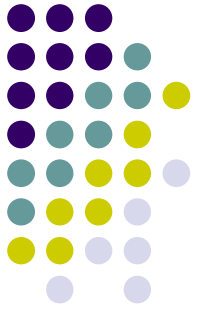




Why does this work?

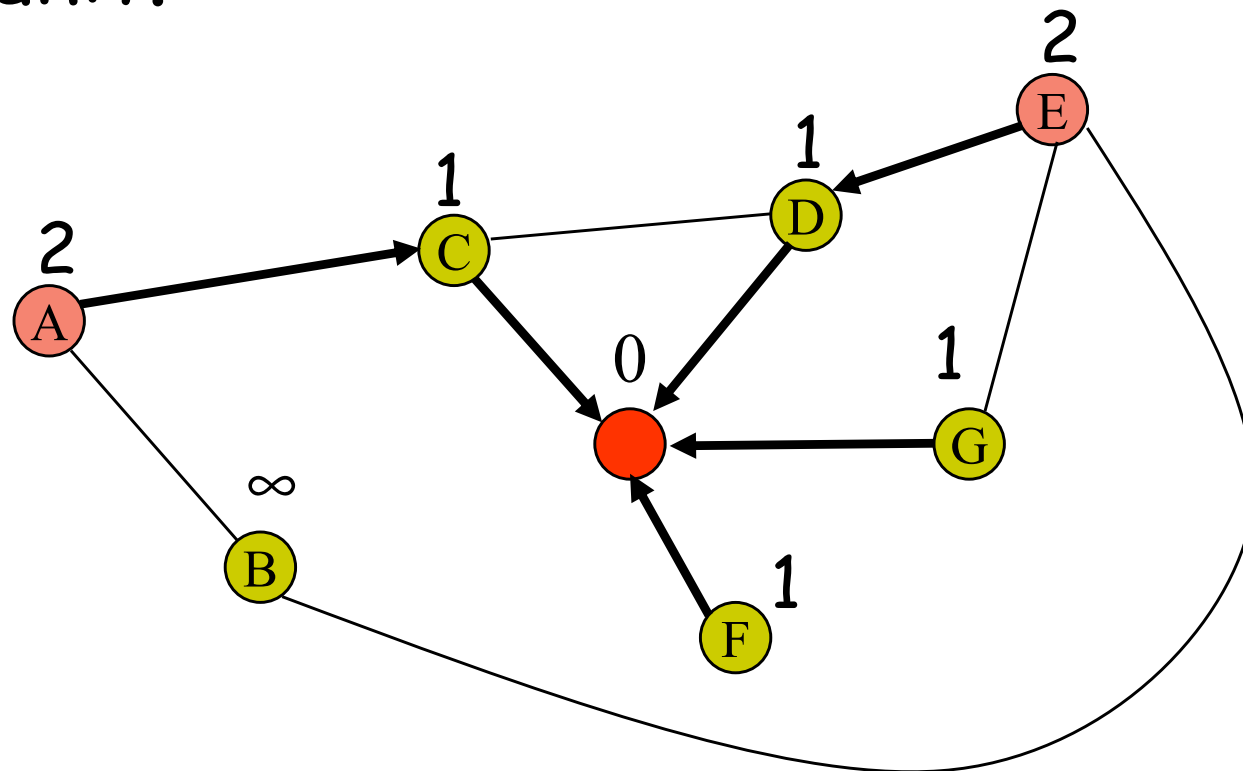
- Suppose all nodes start with distance ∞ , and suppose that updates are sent every time unit.

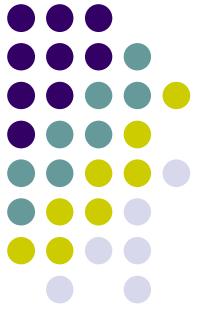




Why does this work?

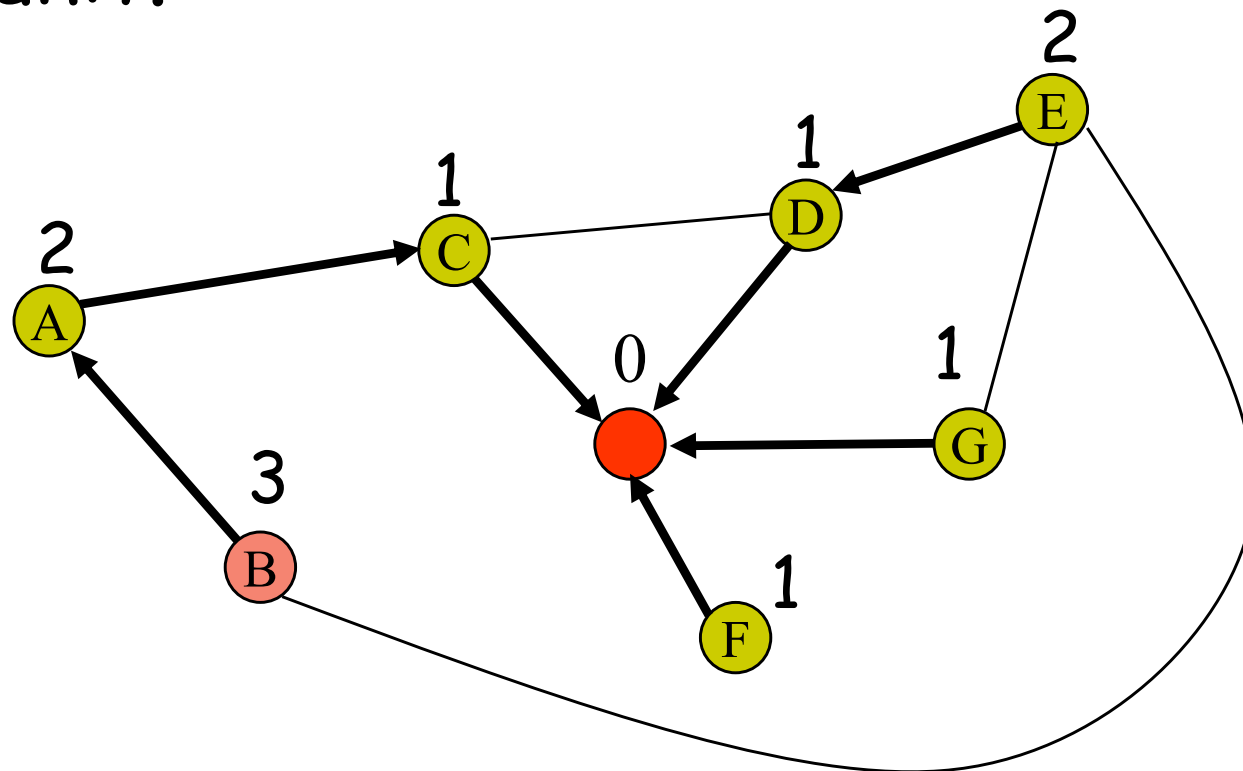
- Suppose all nodes start with distance ∞ , and suppose that updates are sent every time unit.

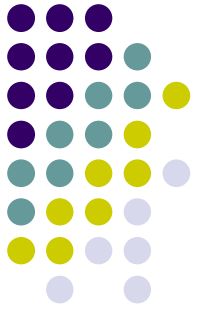




Why does this work?

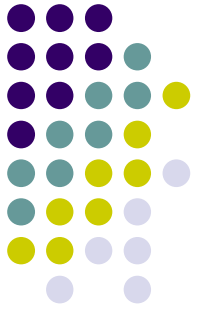
- Suppose all nodes start with distance ∞ , and suppose that updates are sent every time unit.





Bellman-Ford: properties

- Root ports (“parent” pointers) induce a tree
 - Works for any non-negative link weights
- $W(u, v)$:
- 1'. $\text{cost}_v(u) := \text{dist}_u + W(u, v)$
- Works even when the system operates asynchronously.
 - Works regardless of the initial distances! (later...)



3. ST of LAN segments

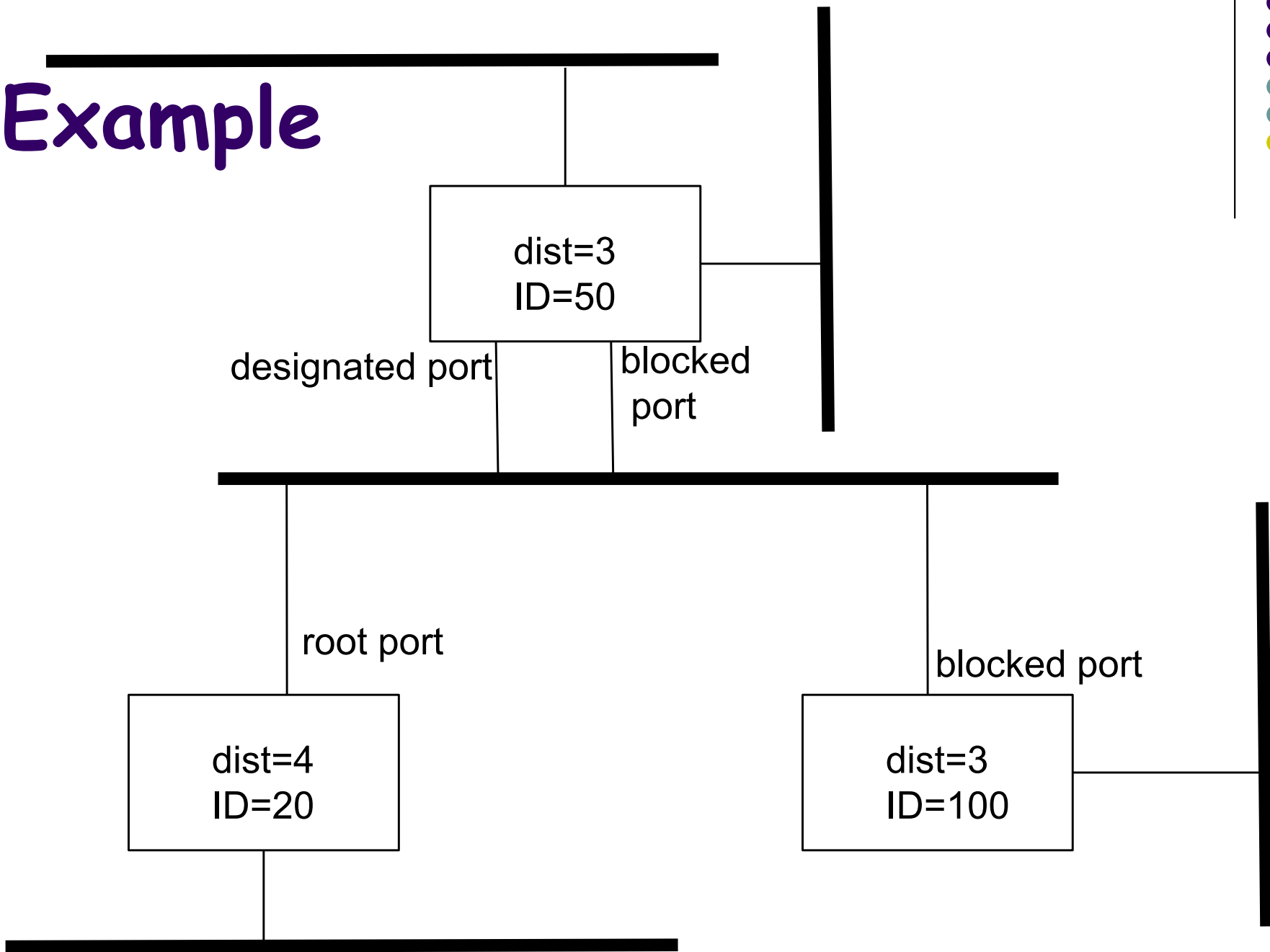
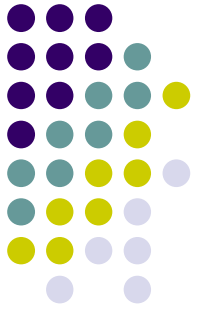
Assumption: given an ST of the bridges

Idea: Each segment “chooses” exactly one attached bridge to forward packets to and from the segment, and others are blocked (as if disconnected)

- Bridge closest to root is selected. Ties broken by bridge ID (and if needed, by port ID)

Implementation: Bridges listen to all ports. Bridge marks port as “**designated port**” iff the bridge is the **best** on that port

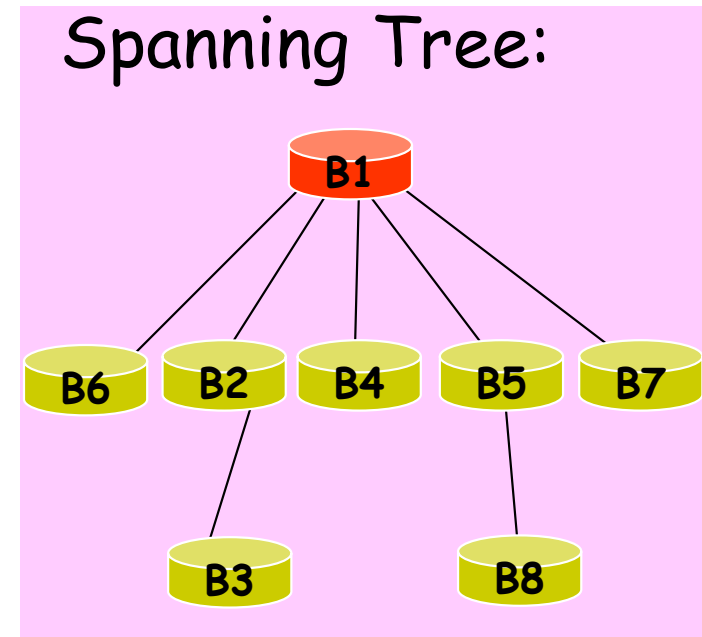
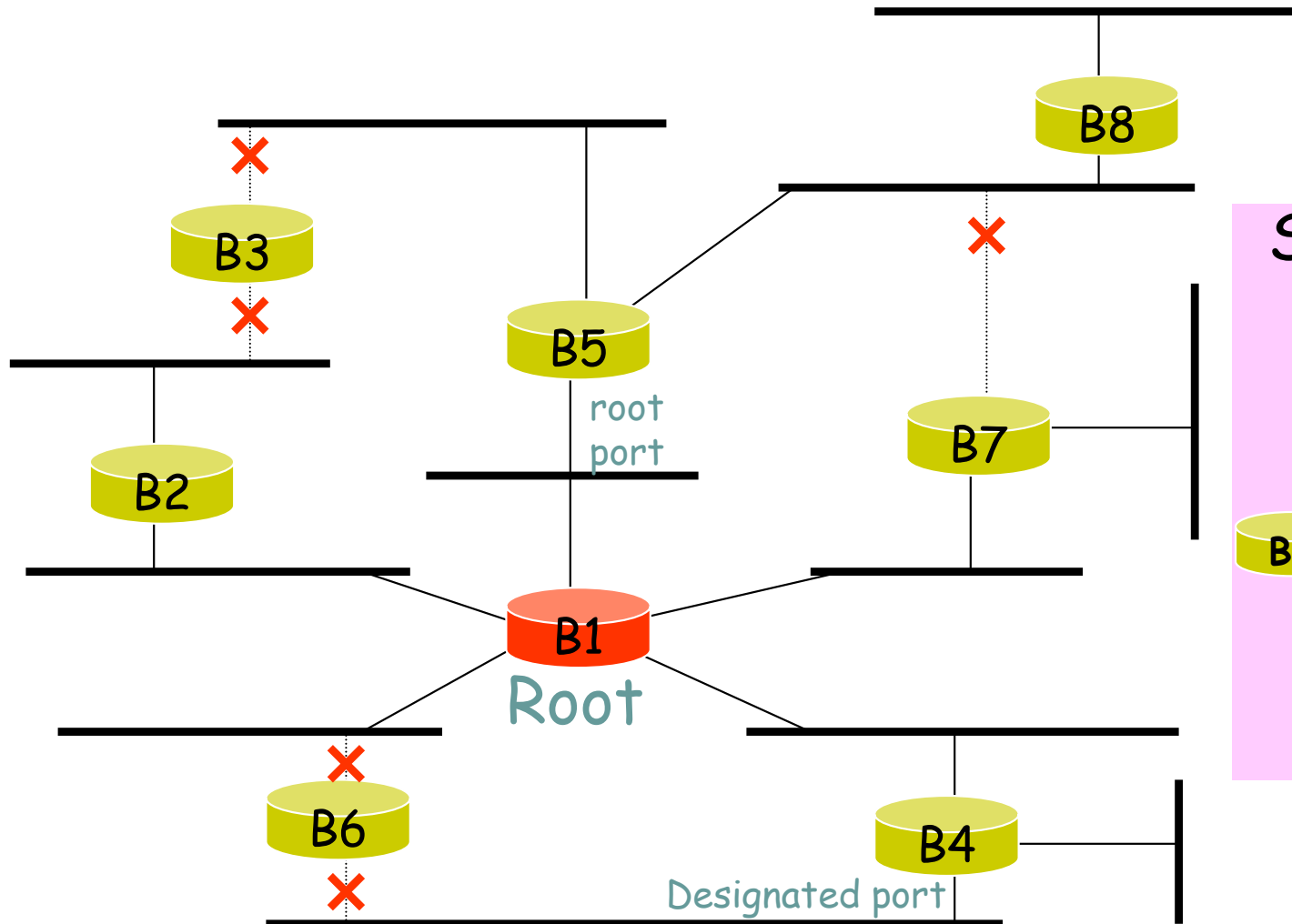
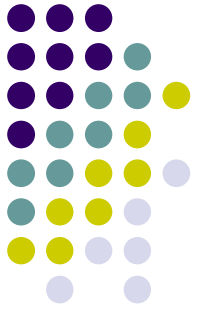
Example



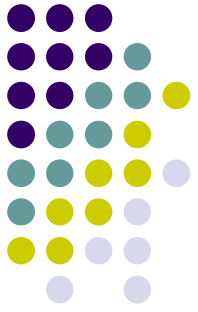


- 34

Example Spanning Tree



Spanning Tree Algorithm: Implementation



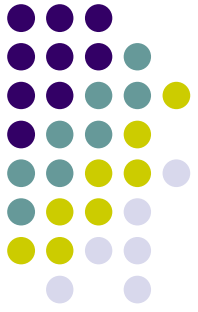
Keep pumping a single message:

`(myRoot, myDist, myID)`

BPDU: Bridge Protocol Data Unit

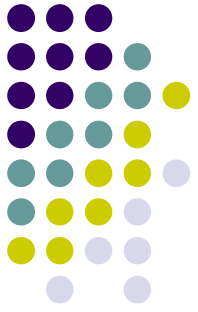
- Store and forward only the **lexicographically-first** BPDU seen so far
- Can never be worse than `(myID, 0, myID)`
- Forget old messages

Spanning Tree Algorithm: Select Designated Ports



- A bridge sends BPDU frames on all its ports, and listens to other BPDUs received
- For each of its ports, bridge determines whether its BPDU is best (by lex. order)
 - If the bridge sends the best BPDU on port p, then p is a **designated port**

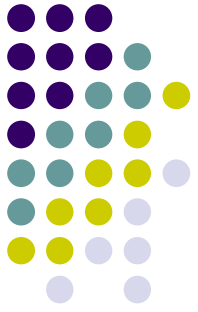
Each segment has exactly one designated port



Forwarding/Blocking State

1. Only root and designated ports are active for forwarding data messages (both ways)
2. All other ports send and listen only to BPDUs messages
 - They are in the *blocking* state: non-existent for data messages!

Algorhyme, by Radia Perlman



I think that I shall never see
a graph more lovely than a tree.

A tree whose crucial property
is loop-free connectivity.

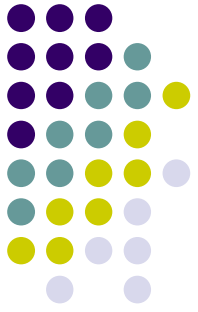
A tree that must be sure to span
so packet can reach every LAN.

First, the root must be selected.
By ID, it is elected.

Least-cost paths from root are traced.
In the tree, these paths are placed.

A mesh is made by folks like me,
then bridges find a spanning tree.





New Concepts

- Repeaters, hubs
 - Learning bridge
 - Bellman-Ford Alg.
 - Spanning Tree Protocol
 - Root port
 - Designated port
 - BPDU
- Blocked/Forwarding port